Introduction to GenAI

LangChain Fundamentals

1. Implement a LangChain chat model that takes a user's query about the weather forecast for a specific city and responds with a JSON object containing the `city_name`, `forecast`, `temperature`, and `timestamp` of the response.

2. Create a LangChain application that accepts a user query about available job positions in a specific industry, uses a prompt template to format the input, and generates a response that is parsed into a Pydantic model with fields for `job_title`, `company_name`, `location`, `salary_range`, and `job_description`.

3. Develop a tool in LangChain that calls an external API to retrieve current cryptocurrency prices based on the user's specified coin (e.g., Bitcoin, Ethereum). Implement the logic to generate the API call with the correct parameters and parse the response to extract `coin_name`, `current_price`, and `price_change_percentage`.

4. Build a LangChain agent that can detect the language of a user's input. If the input is not in English, the agent should call a translation API to translate the text into English. The agent should handle the API call with the correct parameters and return the translated text along with the `source_language`.

5. Write a LangChain script that takes a user's input for booking a hotel room, processes it through a chat model, and formats the output into an XML structure containing elements such as `hotel_name`, `check_in_date`, `check_out_date`, `room_type`, `total_price`, and `confirmation_number`. The output XML should be ready for submission to a hotel booking system.

6. Implement a LangChain chat model that generates a structured JSON output for a user asking for restaurant recommendations. The JSON should include `restaurant_name`, `cuisine_type`, `location`, `average_cost`, and `rating` for each recommended restaurant.

7. Develop a LangChain agent that uses tool-calling to interact with a currency conversion API. The agent should determine when to convert currencies based on user input (e.g., "Convert 100 USD to EUR") and return the result in a structured format with fields for `amount_in_usd`, `converted_amount`, `currency`, and `conversion_rate`.

8. Create a LangChain application where the chat model generates a list of tasks based on a user's request to organize a corporate event. Each task should be structured as a JSON object with fields for `task_name`, `assigned_to`, `due_date`, and `priority_level`.

9. Build a LangChain system that chains together a chat model and an output parser to generate a response to a user's request to create a shopping list. The output should be converted into an XML format with elements for each `item_name`, `quantity`, and `category`, ensuring that the XML is well-structured and suitable for import into a shopping app.